



Universidade do Minho
Escola de Engenharia

Departamento de Informática

Licenciatura em Engenharia Informática

Redes de Computadores

Trabalho Prático Nº2: Packet Sniffer

Grupo PL64

Tomás Teles Coelho, nº A111262

Tomás Barroso Ramalhete, nº A98953

Francisco Queirós Ribeiro da Costa Maia, nº A108962

Índice

1	Introdução	1
2	Descrição da Arquitetura do Sniffer	2
2.1	Módulos	2
2.2	Fluxo de Processamento	3
3	Catálogo e Análise de Protocolos	3
3.1	Camada de ligação de dados (L2) e rede (L3)	3
3.2	Camada de transporte (L4)	5
3.3	Camada de aplicação (L7)	5
4	Filtros e Parâmetros de Entrada	7
4.1	Parâmetros de captura e visualização	7
4.2	Motor de filtragem	7
5	Mecanismos de Logging	8
6	Descrição da Topologia no CORE	8
6.1	Arquitetura da rede	9
6.2	Identificação e validação de protocolos	9
7	Testes Efetuados	10
7.1	Teste 1: Correlação de pacotes ICMP (Ping)	10
7.2	Teste 2: Captura contínua e estatísticas (Traceroute)	11
7.3	Teste 3: Filtragem e agrupamento de fragmentação IPv4	11
7.4	Teste 4: Acompanhamento do estado TCP	12
7.5	Teste 5: Mecanismos de Persistência (Logging)	13
7.6	Teste 6: Captura em Interface Física/Real	14
8	Análise na Interface Real e Obstáculos Encontrados	15
9	Limitações da Aplicação Desenvolvida	15
10	Conclusão	16

1 Introdução

O presente relatório descreve o desenvolvimento de um *packet sniffer* no âmbito do segundo trabalho prático da Unidade curricular de Redes de Computadores. O trabalho tem como objetivo o desenvolvimento de uma ferramenta de captura e análise de tráfego de rede, com funcionalidades semelhantes ao *Wireshark*, mas desenvolvida de "raiz" pelo grupo.

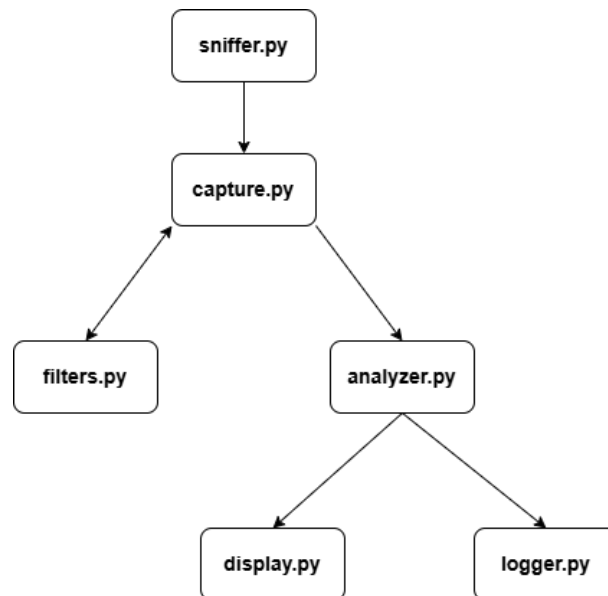
O sniffer foi implementado em **Python**, recorrendo à biblioteca **Scapy** para a captura e *parsing* de pacotes, conforme recomendado no enunciado. A ferramenta é capaz de capturar pacotes em tempo real a partir de uma interface de rede, identificar os protocolos envolvidos, exibir informação relevante na consola, e guardar as capturas em diferentes formatos.

O desenvolvimento foi realizado em duas fases:

1. **Parte A:** captura numa rede emulada no *CORE*, num ambiente controlado e reprodutível;
2. **Parte B:** captura na interface real do computador, permitindo observar tráfego real de protocolos como Wi-Fi (IEEE 802.11) e DHCP

2 Descrição da Arquitetura do Sniffer

2.1 Módulos



Arquitetura modular do packet sniffer

- **Ponto de entrada (`sniffer.py`):** Atua como o controlador principal da aplicação. Recorrendo à biblioteca `argparse`, processa os argumentos da linha de comandos (interface, filtros, opções de *logging*), inicializa os restantes módulos e instancia o motor de captura. Ao finalizar, processa e apresenta as estatísticas globais de tráfego.
- **Motor de captura (`capture.py`):** Contém a classe `SnifferEngine`, responsável por invocar a função `sniff()` da biblioteca *Scapy*. Além de capturar os pacotes brutos, este módulo implementa um mecanismo de gestão de estado fundamental para identificar contextos de rede avançados, como a correlação entre pedidos e respostas ICMP e a identificação e agrupamento de fragmentos IPv4.
- **Analizador de protocolos (`analyzer.py`):** Recebe um pacote bruto do motor de captura e processa as suas camadas de forma hierárquica (L2, L3, L4 e L7). Este módulo transforma a estrutura complexa do *Scapy* num dicionário Python normalizado, contendo sumários legíveis e detalhes extraídos (MACs, IPs, portas, *flags*, etc.).
- **Motor de filtragem (`filters.py`):** Avalia o dicionário gerado pelo analisador contra as condições definidas pelo utilizador (por protocolo, IP, MAC, portas, *flags* TCP ou fragmentação). Funciona como um filtro de alto nível, descartando os

pacotes que não cumprem os critérios definidos antes de serem processados para visualização ou armazenamento.

- **Módulos de saída (`display.py` e `logger.py`):** Funcionam de forma independente, mas podem operar em simultâneo. O `display.py` trata da formatação tabular e colorida (recorrendo à biblioteca `colorama`) para apresentação na consola em tempo real (*modo live*). O `logger.py` assegura a persistência dos pacotes processados no disco rígido, suportando múltiplos formatos de serialização estruturada (CSV, JSON) e legível (TXT).

2.2 Fluxo de Processamento

O processamento de cada pacote segue os seguintes passos:

1. O Scapy captura um pacote na interface e chama a função `_process_packet()` do `SnifferEngine`;
2. O `PacketAnalyzer.analyze()` identifica os protocolos presentes e constrói um dicionário normalizado com todos os campos relevantes;
3. O `FilterEngine.matches()` verifica se o pacote respeita os filtros configurados; caso contrário, é descartado;
4. Se o modo *live* estiver ativo, o `Display` imprime uma linha formatada na consola;
5. Se o `Logger` estiver configurado, o pacote é escrito no ficheiro de log.

3 Catalogação e Análise de Protocolos

A identificação e dissecação dos protocolos é efetuada de forma hierárquica, utilizando o método `haslayer()` da biblioteca *Scapy*. A análise percorre o pacote desde a camada de ligação de dados até à camada de aplicação, extraindo os campos relevantes. Os dados extraídos são juntados num dicionário normalizado, facilitando a sua posterior filtragem e exportação.

3.1 Camada de ligação de dados (L2) e rede (L3)

- **Ethernet e IPv4:** Constituem a base da maioria dos pacotes analisados. O extrator retira os endereços MAC de origem e destino do cabeçalho Ethernet. Ao detetar a camada IPv4, extrai os endereços IP (origem/destino), TTL, *flags* de fragmentação e identificadores (IP ID). No módulo `capture.py`, existe lógica de estado para identificar e correlacionar fragmentos IPv4, agrupando-os visualmente para o utilizador.

- **ARP:** O protocolo ARP (*Address Resolution Protocol*) opera na camada de ligação e é responsável pelo mapeamento entre endereços IP e endereços MAC. A identificação é feita pela presença de uma camada ARP no pacote Scapy, verificada com `packet.haslayer(ARP)`. Os campos analisados incluem: operação (request/reply), MAC e IP de quem enviou, e MAC e IP do destino. Uma troca ARP completa envolve um *broadcast request* seguido de um *unicast reply*. No final, o analisador formata um resumo em linguagem natural (ex: “Quem tem o IP X?”) facilitando a leitura.

```
def _parse_arp(self, packet, r):
    arp = packet[ARP]
    op = 'request' if arp.op == 1 else 'reply'

    r['protocol'] = 'ARP'
    r['src_ip'] = arp.psrc
    r['dst_ip'] = arp.pdst
    r['src_mac'] = arp.hwsrc
    r['dst_mac'] = arp.hwdst

    if op == 'request':
        r['summary'] = "ARP Request: MAC {} pergunta quem tem o IP {}"
    else:
        r['summary'] = "ARP Reply: IP {} esta no MAC {} (Destino: {})"

    r['details'] = {
        'operacao': op,
        'sender_mac': arp.hwsrc,
        'sender_ip': arp.psrc,
        'target_mac': arp.hwdst,
        'target_ip': arp.pdst,
    }
    return r
```

- **ICMP:** O ICMP (*Internet Control Message Protocol*) é identificado quando um pacote IPv4 contém uma camada ICMP. São analisados o tipo (*type*), o código (*code*), o identificador e o número de sequência (para *Echo Request/Reply*), relevantes para identificar trocas do tipo ping.

Tipos ICMP identificados pelo sniffer.

Type	Designação	Contexto típico
0	Echo Reply	Resposta ao ping
3	Destination Unreachable	Host/porta inacessível
8	Echo Request	Comando ping
11	Time Exceeded	TTL expirado (traceroute)

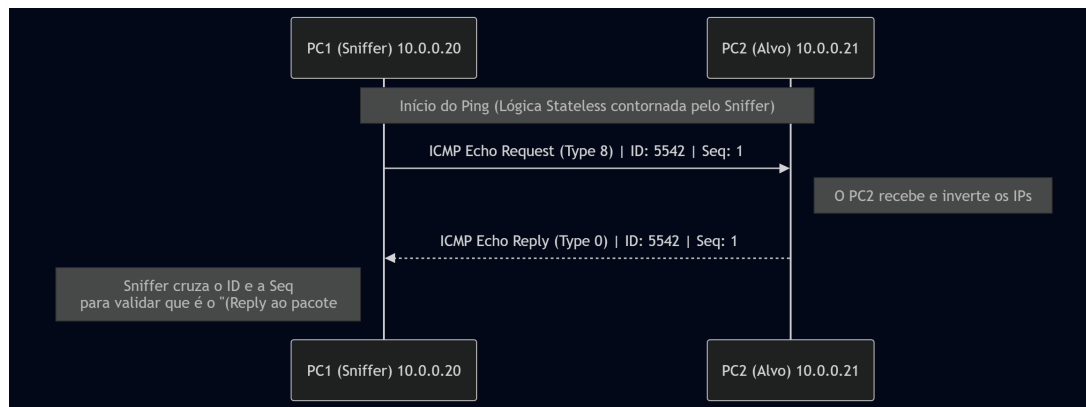


Diagrama de Interação Ping (ICMP)

3.2 Camada de transporte (L4)

- **UDP (User Datagram Protocol):** Identificação básica de portas de origem e destino e o comprimento do *payload*.
- **TCP (Transmission Control Protocol):** O analisador interpreta o campo de *flags* para reconhecer e descrever as fases da ligação TCP (estabelecimento via *Three-way Handshake*, transmissão ou terminio). O excerto de código abaixo, retirado do `analyzer.py`, demonstra como esta análise de contexto é efetuada:

```
if 'S' in flags and 'A' not in flags:
    note = '-'(SYN--inicio-de-ligacao)'
elif 'S' in flags and 'A' in flags:
    note = '-'(SYN-ACK--ligacao-aceite)'
elif 'F' in flags and 'A' in flags:
    note = '-'(FIN-ACK--terminio)'
elif 'R' in flags:
    note = '-'(RST--reset)'
r['summary'] = base + note
```

3.3 Camada de aplicação (L7)

- **DHCP (Dynamic Host Configuration Protocol):** É detetado sempre que o tráfego UDP flui através das portas 67 ou 68. O analisador processa a camada interna para descobrir o tipo de mensagem (*Discover*, *Offer*, *Request*, *ACK*) e extrai dados cruciais oferecidos pelos servidores, como o *lease time* e o IP sugerido.
- **HTTP (Hypertext Transfer Protocol):** Quando existe tráfego TCP nas portas 80 ou 8080, o analisador tenta decodificar o *payload* da camada de aplicação (Raw). Como ilustrado abaixo, procura por verbos HTTP conhecidos (como **GET** ou **POST**) para garantir que se trata efetivamente de tráfego HTTP antes de processar o cabeçalho:

```

if packet.haslayer(Raw):
    payload = packet[Raw].load
    http_verbs = (b'GET', b'POST', b'PUT', b'DELETE', b'HTTP/')
if any(payload.startswith(v) for v in http_verbs):
    first_line = payload.decode('utf-8', errors='replace').split('\r\n')[0]
    r['summary'] = f"HTTP-{first_line}"

```

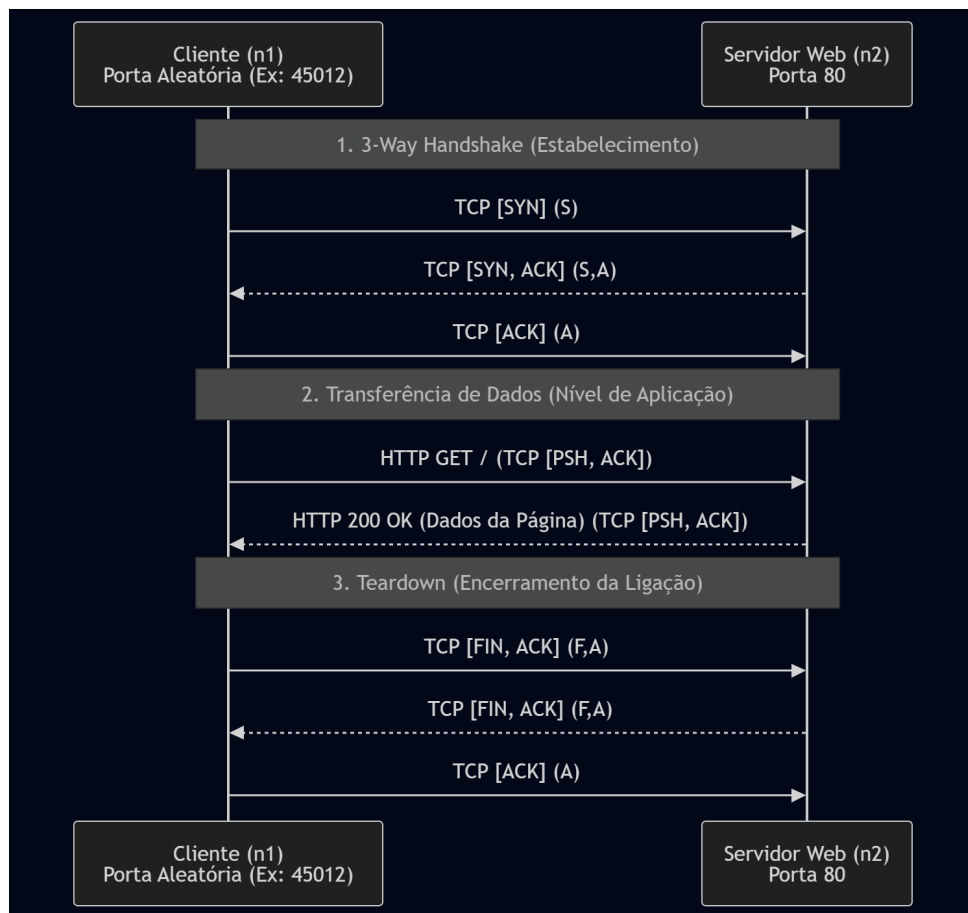


Diagrama de Interação HTTP

4 Filtros e Parâmetros de Entrada

A configuração é feita via interface de linha de comandos, recorrendo à biblioteca `argparse`. Os parâmetros dividem-se em configuração de captura, filtros de alto nível e filtros de baixo nível.

4.1 Parâmetros de captura e visualização

- `-i` ou `--interface`: Define a interface de rede a escutar (ex: `eth0`). É de preenchimento obrigatório.
- `--list-interfaces`: Auxiliar que lista as interfaces disponíveis no sistema e os seus respetivos IPs.
- `-c` ou `--count`: Define um limite máximo de pacotes a capturar (útil para amostragens controladas).
- `-v` ou `--verbose`: Expande a visualização na consola, exibindo o dicionário de detalhes completos de cada pacote (útil para análise profunda).
- `--no-live`: Suprime totalmente a impressão dos pacotes na consola, desativando o módulo de visualização.

4.2 Motor de filtragem

Implementado no módulo `filters.py`, atua após a dissecação do pacote. Caso o pacote não cumpra as condições estabelecidas, é imediatamente descartado. Os filtros suportados são:

- `--protocol`: Filtra tráfego por protocolo específico (ex: `icmp`, `tcp`, `http`). O motor é sensível a hierarquias (ex: o filtro `tcp` aceita pacotes classificados como `http`).
- `--ip`, `--src-ip`, `--dst-ip`: Filtragem por endereço IPv4 exato, distinguindo entre origem, destino ou ambos.
- `--mac`: Filtragem por endereço físico.
- `--port`: Filtra comunicações cujo porto TCP ou UDP (origem ou destino) corresponda ao valor indicado.
- `--tcp-flags`: Permite filtrar pacotes com base nas *flags* ativas (ex: `S` para SYN, `FA` para FIN-ACK).
- `--fragmented`: Isola pacotes IPv4 que possuam a *flag More Fragments* ou um *offset* superior a zero, ignorando tráfego não fragmentado.

5 Mecanismos de Logging

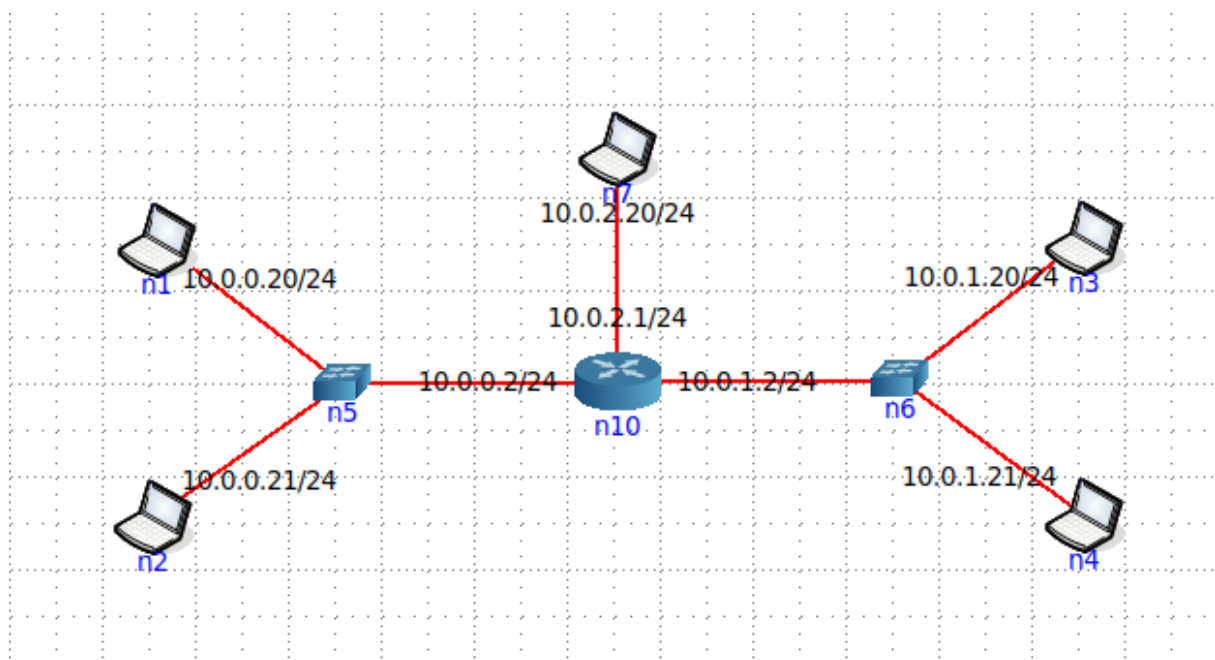
O armazenamento do tráfego capturado é assegurado pelo módulo `logger.py`. O requisito de coexistência entre visualização *live* e gravação é cumprido, pois a classe `Logger` pode operar em paralelo com a consola, escrevendo os dados em disco em tempo real. O ficheiro de destino é definido pela *flag* `--log`.

Para garantir a compatibilidade dos dados capturados com ferramentas externas de análise (como o Excel ou sistemas análise estatística), a aplicação suporta três formatos de exportação geridos pela *flag* `--log-format`:

- **CSV (*Comma-Separated Values*)**: Formato por defeito. Escreve uma linha por pacote com colunas estáticas (MAC, IP, portas, tamanho, sumário). Ideal para análise tabular ou importação em folhas de cálculo.
- **JSON (*JavaScript Object Notation*)**: Formato estruturado e hierárquico. Para além dos campos presentes no CSV, serializa também o sub-dicionário de *detalhes* de cada pacote, preservando toda a informação gerada pelo analisador.
- **TXT**: Um ficheiro de texto puro, que replica a organização formatada da consola, útil para armazenamento de longo prazo sem dependência de *software* de *parsing*.

6 Descrição da Topologia no CORE

Para a Fase A do projeto, foi desenhada uma topologia no emulador CORE que simula uma rede corporativa interligada:



Topologia utilizada

6.1 Arquitetura da rede

A rede é centralizada por um *router* (n10), que interliga três sub-redes distintas:

- **Sub-rede Esquerda (10.0.0.0/24):** O *router* liga-se, através do IP 10.0.0.2, ao *switch* n5. A este *switch* estão conectados dois computadores: n1 (10.0.0.20) e n2 (10.0.0.21).
- **Sub-rede Direita (10.0.1.0/24):** O *router* liga-se, através do IP 10.0.1.2, ao *switch* n6. Neste domínio local encontram-se os computadores n7 (10.0.1.20) e n4 (10.0.1.21).
- **Sub-rede Superior (10.0.2.0/24):** O computador n3 (10.0.2.20) encontra-se ligado diretamente à interface 10.0.2.1 do *router*.

6.2 Identificação e validação de protocolos

Esta divisão em três sub-redes permite desencadear e capturar as interações protocolares necessárias para mostrar o funcionamento correto do projeto:

- **ARP:** A existência de *switches* (n5 e n6) interligando múltiplas máquinas permite observar a resolução de endereços MAC locais. Ao efetuar um *ping* do n1 para o n2, o *sniffer* em execução captura os pacotes *ARP Request* (em *broadcast*) e os *ARP Replies* (em *unicast*) trocados dentro da mesma sub-rede.
- **ICMP e IPv4:** A comunicação entre nós de sub-redes diferentes (por exemplo, de n1 para n4) obriga o tráfego a atravessar o *router* n10. Isto permite testar o *sniffer* na captura de pacotes *Echo Request* e *Echo Reply*, mostrando assim a sua capacidade de associar o ID e a sequência da resposta ao pedido original.
- **TCP e HTTP:** O nó n3, estando isolado na sua sub-rede, serve como ambiente ideal para instanciar um servidor HTTP. Ao utilizar o comando `curl` a partir do nó n7 direcionado ao n3, o *sniffer* consegue capturar a totalidade da sessão TCP de ponta a ponta: o *Three-way handshake* (SYN, SYN-ACK, ACK), a transferência do *payload* HTTP e o fim da ligação (FIN, FIN-ACK).
- **UDP e DHCP:** Os domínios de *broadcast* delimitados pelos *switches* n5 e n6 permitem instanciar um servidor DHCP local e capturar as trocas UDP (*Discover*, *Offer*, *Request*, *ACK*) sem que estas inundem as restantes sub-redes, isolando o tráfego de controlo.

7 Testes Efetuados

Para garantir que todos os requisitos funcionais foram cumpridos, foram executados vários testes sobre a topologia emulada no CORE e na máquina real. Estes testes visaram estimular o motor de captura, os filtros e a gestão de estado.

7.1 Teste 1: Correlação de pacotes ICMP (Ping)

Envio de pedidos *ICMP Echo Request* do nó N7 (10.0.1.20) para o nó N1 (10.0.0.20), usando o comando: `python3 sniffer.py -i eth0 --ip 10.0.1.20 -v`

```
root@n1:/home/core/core/Wireshark-Caseiro-Final# python3 sniffer.py -i eth0 --ip 10.0.1.20 -v
#      Tempo(s)  Proto  Origem      Destino      Bytes  Resumo
=====
RC Packet Sniffer | interface: eth0
Ctrl+C para parar
1  7.266  ICMP   10.0.1.20    10.0.0.20    98     ICMP Echo Request 10.0.1.20 -> 10.0.0.20 id=30 seq=1
    ↳ ttl: 63
    ↳ ip_id: 46997
    ↳ flags: DF
    ↳ tos: 0
    ↳ type: 8
    ↳ type_str: Echo Request
    ↳ code: 0
    ↳ icmp_id: 30
    ↳ icmp_seq: 1
2  7.267  ICMP   10.0.0.20    10.0.1.20    98     ICMP Echo Reply 10.0.0.20 -> 10.0.1.20 id=30 seq=1 (Reply ao pacote #1)
    ↳ ttl: 64
    ↳ ip_id: 21239
    ↳ flags:
    ↳ tos: 0
    ↳ type: 0
    ↳ type_str: Echo Reply
    ↳ code: 0
    ↳ icmp_id: 30
    ↳ icmp_seq: 1
3  8.300  ICMP   10.0.1.20    10.0.0.20    98     ICMP Echo Request 10.0.1.20 -> 10.0.0.20 id=30 seq=2
    ↳ ttl: 63
    ↳ ip_id: 47112
    ↳ flags: DF
    ↳ tos: 0
    ↳ type: 8
    ↳ type_str: Echo Request
    ↳ code: 0
    ↳ icmp_id: 30
    ↳ icmp_seq: 2
4  8.301  ICMP   10.0.0.20    10.0.1.20    98     ICMP Echo Reply 10.0.0.20 -> 10.0.1.20 id=30 seq=2 (Reply ao pacote #3)
    ↳ ttl: 64
    ↳ ip_id: 21314
    ↳ flags:
    ↳ tos: 0
    ↳ type: 0
    ↳ type_str: Echo Reply
    ↳ code: 0
    ↳ icmp_id: 30
    ↳ icmp_seq: 2
5  9.321  ICMP   10.0.1.20    10.0.0.20    98     ICMP Echo Request 10.0.1.20 -> 10.0.0.20 id=30 seq=3
    ↳ ttl: 63
    ↳ ip_id: 47332
    ↳ flags: DF
    ↳ tos: 0
    ↳ type: 8
    ↳ type_str: Echo Request
    ↳ code: 0
    ↳ icmp_id: 30
    ↳ icmp_seq: 3
6  9.321  ICMP   10.0.0.20    10.0.1.20    98     ICMP Echo Reply 10.0.0.20 -> 10.0.1.20 id=30 seq=3 (Reply ao pacote #5)
    ↳ ttl: 64
    ↳ ip_id: 21408
    ↳ flags:
    ↳ tos: 0
    ↳ type: 0
    ↳ type_str: Echo Reply
    ↳ code: 0
    ↳ icmp_id: 30
```

Teste 1

Este teste validou simultaneamente o filtro por endereço IP e a gestão de estado do *sniffer*. Através do modo *verbose*, foi possível observar a extração detalhada dos campos `id` e `seq`. A ferramenta identificou com sucesso o *Echo Request* e, ao intercetar o *Echo Reply*, injetou no sumário a *tag* (Reply ao pacote #X), comprovando a capacidade de emparelhamento da aplicação.

7.2 Teste 2: Captura contínua e estatísticas (Traceroute)

Execução de um *traceroute* do nó N7 para o N1 com o comando no N1: `python3 sniffer.py -i eth0 --ip 10.0.1.20 -v`

```
14 16.910 UDP 10.0.1.20:37466 10.0.0.20:33444 74 UDP 10.0.1.20:37466 -> 10.0.0.20:33444 len=40
    ↳ ttl: 3
    ↳ ip_id: 36440
    ↳ flags:
    ↳ tos: 0
    ↳ src_port: 37466
    ↳ dst_port: 33444
    ↳ length: 40
15 16.911 UDP 10.0.1.20:51897 10.0.0.20:33445 74 UDP 10.0.1.20:51897 -> 10.0.0.20:33445 len=40
    ↳ ttl: 3
    ↳ ip_id: 36441
    ↳ flags:
    ↳ tos: 0
    ↳ src_port: 51897
    ↳ dst_port: 33445
    ↳ length: 40
16 16.911 UDP 10.0.1.20:60343 10.0.0.20:33446 74 UDP 10.0.1.20:60343 -> 10.0.0.20:33446 len=40
    ↳ ttl: 4
    ↳ ip_id: 36442
    ↳ flags:
    ↳ tos: 0
    ↳ src_port: 60343
    ↳ dst_port: 33446
    ↳ length: 40
17 16.911 UDP 10.0.1.20:38122 10.0.0.20:33447 74 UDP 10.0.1.20:38122 -> 10.0.0.20:33447 len=40
    ↳ ttl: 4
    ↳ ip_id: 36443
    ↳ flags:
    ↳ tos: 0
    ↳ src_port: 38122
    ↳ dst_port: 33447
    ↳ length: 40
18 16.912 UDP 10.0.1.20:58013 10.0.0.20:33448 74 UDP 10.0.1.20:58013 -> 10.0.0.20:33448 len=40
    ↳ ttl: 4
    ↳ ip_id: 36444
    ↳ flags:
    ↳ tos: 0
    ↳ src_port: 58013
    ↳ dst_port: 33448
    ↳ length: 40
19 16.912 UDP 10.0.1.20:50535 10.0.0.20:33449 74 UDP 10.0.1.20:50535 -> 10.0.0.20:33449 len=40
    ↳ ttl: 5
    ↳ ip_id: 36445
    ↳ flags:
    ↳ tos: 0
    ↳ src_port: 50535
    ↳ dst_port: 33449
    ↳ length: 40
^C
=====
RESUMO DA CAPTURA
=====
Início: 2026-05-02 13:32:58 | Duração: 38.36s | Total Pacotes: 19
Hierarchy Statistics (Taxas Médias & Totais):
Protocolo | Pacotes | % Pkts | Bytes | Tx (Pkts/s) | Tx (Bytes/s)
-----
Ethernet | 19 | 100.0% | 1.5 KB | 0.50 | 41.0 B/s
├── IPv4 | 19 | 100.0% | 1.5 KB | 0.50 | 41.0 B/s
├── UDP | 13 | 68.4% | 962.0 B | 0.34 | 25.1 B/s
└── ICMP | 6 | 31.6% | 612.0 B | 0.16 | 16.0 B/s
```

```
vcmd
64 bytes from 10.0.0.20: icmp_seq=2 ttl=63 time=0.231 ms
64 bytes from 10.0.0.20: icmp_seq=3 ttl=63 time=0.146 ms
64 bytes from 10.0.0.20: icmp_seq=4 ttl=63 time=0.101 ms
64 bytes from 10.0.0.20: icmp_seq=5 ttl=63 time=0.244 ms
64 bytes from 10.0.0.20: icmp_seq=6 ttl=63 time=0.117 ms
^C
--- 10.0.0.20 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 512ms
rtt min/avg/max/ndev = 0.101/0.189/0.286/0.071 ms
root@n7:/tmp/pcapcore.45319/n7_conf# ping 10.0.0.20
PING 10.0.0.20 (10.0.0.20) 56(84) bytes of data:
64 bytes from 10.0.0.20: icmp_seq=1 ttl=63 time=0.097 ms
64 bytes from 10.0.0.20: icmp_seq=2 ttl=63 time=0.124 ms
64 bytes from 10.0.0.20: icmp_seq=3 ttl=63 time=0.245 ms
64 bytes from 10.0.0.20: icmp_seq=4 ttl=63 time=0.118 ms
^C
--- 10.0.0.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3090ms
rtt min/avg/max/ndev = 0.097/0.136/0.245/0.068 ms
root@n7:/tmp/pcapcore.45319/n7_conf# traceroute 10.0.0.20
traceroute to 10.0.0.20 (10.0.0.20): 30 hops max, 60 byte packets
 1 10.0.1.2 (10.0.1.2) 0.184 ms 0.006 ms 0.003 ms
 2 10.0.0.20 (10.0.0.20) 0.077 ms 0.013 ms 0.007 ms
root@n7:/tmp/pcapcore.45319/n7_conf#
```

Teste 2

O *traceroute* gera pacotes UDP com *Time-To-Live* (TTL) incremental, originando respostas ICMP ao longo do percurso. O teste mostra a correta dissecação da camada de transporte (UDP). Após interromper a captura, a aplicação exibiu corretamente a tabela final de *Hierarchy Statistics*, calculando a percentagem de pacotes, os volumes transferidos e as taxas de débito (*Pkts/s* e *Bytes/s*).

7.3 Teste 3: Filtragem e agrupamento de fragmentação IPv4

Envio de um *ping* do nó N5 para o N1 (`ping -c 1 -s 4000 10.0.0.20`), executando também o comando no N1: `python3 sniffer.py -i eth0 --fragmented -v`

```
root@n1:/volume/RC/Wireshark-Caseiro-main# python3 sniffer.py -i eth0 --fragmented -v
# Tempo(s)  Proto  Origem  Destino  Bytes  Resumo
=====
Ctrl+C para parar
1  18,582  Icmp    10.0.1.21  10.0.0.20  1514  IPv4 Fragmento (id=36736 offset=0) Icmp Echo Request 10.0.1.21 + 10.0.0.20 id=15745 seq=1 [Início da Fragmentação]
   k.ttl: 63
   k.ip_id: 36736
   k.flags: MF
   k.tos: 0
   k.type: 0
   k.code: 0
   k.sequence: 15745
   k.sequence: 1
2  18,585  Icmp    10.0.1.21  10.0.0.20  1514  IPv4 Fragmento (id=36736 offset=185) IPv4 10.0.1.21 + 10.0.0.20 protocol [Fragmento 2 associado ao #1]
   k.ttl: 63
   k.ip_id: 36736
   k.flags: MF
   k.tos: 0
   k.type: 0
   k.code: 0
   k.sequence: 15745
   k.sequence: 1
3  18,588  Icmp    10.0.1.21  10.0.0.20  1514  IPv4 Fragmento (id=36736 offset=370) IPv4 10.0.1.21 + 10.0.0.20 protocol [Fragmento 3 associado ao #1, #2]
   k.ttl: 63
   k.ip_id: 36736
   k.flags: MF
   k.tos: 0
   k.type: 0
   k.code: 0
   k.sequence: 15745
   k.sequence: 1
4  18,595  Icmp    10.0.0.20  10.0.1.21  1514  IPv4 Fragmento (id=3432 offset=0) Icmp Echo Reply 10.0.0.20 + 10.0.1.21 id=15745 seq=1 [Resposta ao pacote #1] [Início da Fragmentação]
   k.ttl: 64
   k.ip_id: 3432
   k.flags: MF
   k.tos: 0
   k.type: 0
   k.code: 0
   k.sequence: 15745
   k.sequence: 1
5  18,598  Icmp    10.0.0.20  10.0.1.21  1514  IPv4 Fragmento (id=3432 offset=185) IPv4 10.0.0.20 + 10.0.1.21 protocol [Fragmento 2 associado ao #4]
   k.ttl: 64
   k.ip_id: 3432
   k.flags: MF
   k.tos: 0
   k.type: 0
   k.code: 0
   k.sequence: 15745
   k.sequence: 1
6  18,598  Icmp    10.0.0.20  10.0.1.21  1514  IPv4 Fragmento (id=3432 offset=370) IPv4 10.0.0.20 + 10.0.1.21 protocol [Fragmento 3 associado ao #4, #5]
   k.ttl: 64
   k.ip_id: 3432
   k.flags: MF
   k.tos: 0
   k.type: 0
   k.code: 0
   k.sequence: 15745
   k.sequence: 1
```

Teste 3

Ao injetar um pacote de 4000 *bytes*, forçou-se a rede a fragmentar o tráfego IPv4 (uma vez que excede a MTU padrão de 1500 *bytes*). O uso da *flag* `--fragmented` permite isolar apenas os pacotes com a *flag* *More Fragments* ou *offset* > 0. A aplicação sinalizou corretamente o [Início da Fragmentação] e associou os fragmentos subsequentes ao pacote original através do *ip_id*.

7.4 Teste 4: Acompanhamento do estado TCP

Início de um servidor local no N4 (`python3 -m http.server 4444`) e acesso via N1 através de `telnet 10.0.1.21 4444`. No nó N1, executamos o comando: `python3 sniffer.py -i eth0 --port 4444`

```
root@n1:/volume/RC/Wireshark-Caseiro-main# python3 sniffer.py -i eth0 --port 4444
# Tempo(s)  Proto  Origem  Destino  Bytes  Resumo
=====
RC Packet Sniffer  I interface: eth0
Ctrl+C para parar
1  202,548  TCP    10.0.0.20:49634  10.0.1.21:4444  74  TCP 10.0.0.20:49634 + 10.0.1.21:4444 [S] + SYN (início de ligação)
2  202,549  TCP    10.0.1.21:4444  10.0.0.20:49634  74  TCP 10.0.1.21:4444 + 10.0.0.20:49634 [S,A] + SYN-ACK (ligação aceita)
3  202,550  TCP    10.0.0.20:49634  10.0.1.21:4444  66  TCP 10.0.0.20:49634 + 10.0.1.21:4444 [A]
4  217,054  TCP    10.0.0.20:49634  10.0.1.21:4444  79  TCP 10.0.0.20:49634 + 10.0.1.21:4444 [P,A]
5  217,055  TCP    10.0.1.21:4444  10.0.0.20:49634  66  TCP 10.0.1.21:4444 + 10.0.0.20:49634 [A]
6  217,056  TCP    10.0.1.21:4444  10.0.0.20:49634  570  TCP 10.0.1.21:4444 + 10.0.0.20:49634 [P,A]
7  217,057  TCP    10.0.0.20:49634  10.0.1.21:4444  66  TCP 10.0.0.20:49634 + 10.0.1.21:4444 [A]
8  217,058  TCP    10.0.1.21:4444  10.0.0.20:49634  66  TCP 10.0.1.21:4444 + 10.0.0.20:49634 [F,A] + FIN-ACK (término)
9  217,059  TCP    10.0.0.20:49634  10.0.1.21:4444  66  TCP 10.0.0.20:49634 + 10.0.1.21:4444 [F,A] + FIN-ACK (término)
10 217,059  TCP    10.0.1.21:4444  10.0.0.20:49634  66  TCP 10.0.1.21:4444 + 10.0.0.20:49634 [A]
```

```
root@n5:/volume/RC/Wireshark-Caseiro-main# python3 -m http.server 4444
Serving HTTP on 0.0.0.0 port 4444 (http://0.0.0.0:4444/) ...
10.0.0.20 -- [02/May/2026 17:00:57] code 400, message Bad HTTP/0.9 request type ('0!á!')
10.0.0.20 -- [02/May/2026 17:00:57] "0!á! mundo!" 400 -
```

```
root@n1:/volume/RC/Wireshark-Caseiro-main# telnet 10.0.1.21 4444
Trying 10.0.1.21...
Connected to 10.0.1.21.
Escape character is '^'.
0!á mundo!
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
    <title>Error response</title>
  </head>
  <body>
    <h1>Error response</h1>
    <p>Error code: 400</p>
    <p>Message: Bad HTTP/0.9 request type ('0!á!').</p>
    <p>Error code explanation: HTTPStatus.BAD_REQUEST - Bad request syntax or unsupported method.</p>
  </body>
</html>
Connection closed by foreign host.
root@n1:/volume/RC/Wireshark-Caseiro-main#
```

Teste 4

7.5 Teste 5: Mecanismos de Persistência (Logging)

```
root@kali: /volume/RC/Wireshark-Caseroim-main# python3 sniff.py -i eth0 --log-statistics-test.csv
  Tempo(s)  Protocolo  Origem          Destino          Bytes  Resumo

--- Packet Sniffer / interface: eth0
Log File = statistics-test.csv [CSV]
Ctrl+C para parar

1 1.8953 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
2 1.9624 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
3 5.854 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
4 7.366 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 Ethernet 00:00:00:aa:00:02 > 33:33:00:00:00:05
5 8.864 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
6 9.896 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
7 11.857 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
8 13.867 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
9 15.859 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
10 17.277 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 Ethernet 00:00:00:aa:00:02 > 33:33:00:00:00:05
11 18.293 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
12 19.283 ICMP 10.0.1.21 10.0.0.20 98 ICMP Echo Request 10.0.1.21 > 10.0.0.20 id=24162 seq=1
13 19.324 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 ICMP Echo Reply 10.0.1.21 > 10.0.1.21 id=24162 seq=1 (Reply ao pacote #12)
14 19.369 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
15 20.300 ICMP 10.0.1.21 10.0.0.20 98 ICMP Echo Request 10.0.1.21 > 10.0.0.20 id=24162 seq=2
16 20.324 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 ICMP Echo Reply 10.0.0.20 > 10.0.1.21 id=24162 seq=2 (Reply ao pacote #15)
17 21.324 ICMP 10.0.1.21 10.0.0.20 98 ICMP Echo Request 10.0.1.21 > 10.0.0.20 id=24162 seq=3
18 21.325 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 ICMP Echo Reply 10.0.0.20 > 10.0.1.21 id=24162 seq=3 (Reply ao pacote #17)
19 21.369 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
20 23.860 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
21 24.523 ARP 10.0.0.20 10.0.0.1 42 ARP Request: MAC 00:00:00:aa:00:00 (origem) pergunta pelo dono do IP 10.0.0.1
22 24.524 ARP 10.0.0.1 10.0.0.20 42 ARP Reply: MAC 00:00:00:aa:00:02 (origem) pergunta pelo dono do IP 10.0.0.1
23 24.525 ARP 10.0.0.20 10.0.0.1 42 ARP Reply: MAC 00:00:00:aa:00:00 (origem) responde ao MAC 00:00:00:aa:00:02 (destino) dizendo que tem o IP 10.0.0.1
24 24.526 ARP 10.0.0.1 10.0.0.20 42 ARP Reply: MAC 00:00:00:aa:00:02 (origem) responde ao MAC 00:00:00:aa:00:00 (destino) dizendo que tem o IP 10.0.0.1
25 25.860 IPv4 10.0.0.1 224.0.0.5 78 Ethernet 00:00:00:aa:00:02 > 33:33:00:00:00:05
26 27.416 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 Ethernet 00:00:00:aa:00:02 > 33:33:00:00:00:05
27 27.461 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
28 28.861 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
29 31.862 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
30 32.680 TCP 10.0.1.21:57488 10.0.0.20:80 74 TCP 10.0.1.21:57488 > 10.0.0.20:80 [S] > SYN (início de ligação)
31 32.681 TCP 10.0.1.21:57488 10.0.0.20:80 54 TCP 10.0.0.20:80 > 10.0.1.21:57488 [R] > RST (reset)
32 33.862 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
33 34.863 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
34 37.214 ICMP 10.0.1.21 10.0.0.20 98 ICMP Echo Request 10.0.1.21 > 10.0.0.20 id=41598 seq=1
35 37.215 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 ICMP Echo Reply 10.0.0.20 > 10.0.1.21 id=41598 seq=1 (Reply ao pacote #34)
36 37.258 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 Ethernet 00:00:00:aa:00:02 > 33:33:00:00:00:05
37 37.863 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
38 38.220 ICMP 10.0.1.21 10.0.0.20 98 ICMP Echo Request 10.0.1.21 > 10.0.0.20 id=41598 seq=2
39 38.221 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 ICMP Echo Reply 10.0.0.20 > 10.0.1.21 id=41598 seq=2 (Reply ao pacote #38)
40 39.244 ICMP 10.0.1.21 10.0.0.20 98 ICMP Echo Request 10.0.1.21 > 10.0.0.20 id=41598 seq=3
41 39.245 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 ICMP Echo Reply 10.0.0.20 > 10.0.1.21 id=41598 seq=3 (Reply ao pacote #40)
42 39.864 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
43 41.864 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
44 43.864 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
45 45.866 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
46 47.399 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 Ethernet 00:00:00:aa:00:02 > 33:33:00:00:00:05
47 47.399 Ethernet 00:00:00:aa:00:02 33:33:00:00:00:05 90 Ethernet 00:00:00:aa:00:02 > 33:33:00:00:00:05
48 49.811 Ethernet 00:00:00:aa:00:00 33:33:00:00:00:02 70 Ethernet 00:00:00:aa:00:00 > 33:33:00:00:00:02
49 49.867 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
50 49.867 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
51 53.869 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89
52 55.868 IPv4 10.0.0.1 224.0.0.5 78 IPv4 10.0.0.1 > 224.0.0.5 proto=89

RESUMO DA CAPTURA
Início: 2026-05-02 17:05:38 | Duração: 56.45s | Total Pacotes: 52

Hierarchy Statistics (Taxas Médias e Totais):
Protocolo | Pacotes | % Pkts | Bytes | Tx (Pkts/s) | Tx (Bytes/s)
Ethernet | 52 | 100.0% | 4.1 KB | 0.92 | 80.8 B/s
├── IPv4 | 42 | 80.8% | 3.4 KB | 0.74 | 61.8 B/s
├── ICMP | 10 | 19.2% | 1.1 KB | 0.22 | 20.8 B/s
├── TCP | 2 | 3.8% | 12
```

```

1 index,timestamp,relative_time,interface,protocol,src_mac,dst_mac,src_ip,dst_ip,src_port,dst_port,size,summary
2 1.2026-05-02 17:03:40.286,7.854948,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
3 2.2026-05-02 17:03:40.287,3.854372,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
4 3.2026-05-02 17:03:40.288,7.854948,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
5 4.2026-05-02 17:03:40.289,7.854372,eth0,Ethernet,00:00:aa:00:02:33:30:00:00:05...,90:Ethernet 00:00:aa:00:02:33:30:00:05
6 5.2026-05-02 17:03:40.288,7.854948,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
7 6.2026-05-02 17:03:40.289,9.859821,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
8 7.2026-05-02 17:03:40.290,11.856668,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
9 8.2026-05-02 17:03:40.291,11.856668,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
10 9.2026-05-02 17:03:40.291,15.853177,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
11 10.2026-05-02 17:03:55.810,17.377138,eth0,Ethernet,00:00:aa:00:02:33:30:00:00:05...,90:Ethernet 00:00:aa:00:02:33:30:00:05
12 11.2026-05-02 17:03:56.217.058986,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
13 12.2026-05-02 17:03:57.716.1282822,eth0,ICMP,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.1.21.10.0.20...,90:ICMP Echo Request 10.0.1.21 + 10.0.20
    id=24162 seq=1
14 13.2026-05-02 17:03:58.217.058986,eth0,ICMP,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.1.21.10.0.20...,90:ICMP Echo Reply 10.0.1.21 + 10.0.20
    id=24162 seq1 (Reply ao pacote #12)
15 14.2026-05-02 17:03:58.219.05933,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
16 15.2026-05-02 17:03:58.733.20.297973,eth0,ICMP,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.1.21.10.0.20...,90:ICMP Echo Request 10.0.1.21 + 10.0.20
    id=24162 seq=2
17 16.2026-05-02 17:03:58.734.20.301191,eth0,ICMP,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.1.21.10.0.20...,90:ICMP Echo Reply 10.0.20 + 10.0.1.21
    id=24162 seq=2 (Reply ao pacote #15)
18 17.2026-05-02 17:03:59.757.21.328888,eth0,ICMP,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.1.21.10.0.20...,90:ICMP Echo Request 10.0.1.21 + 10.0.20
    id=24162 seq=3
19 18.2026-05-02 17:03:59.758.21.328888,eth0,ICMP,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.1.21.10.0.20...,90:ICMP Echo Reply 10.0.20 + 10.0.1.21
    id=24162 seq=3 (Reply ao pacote #17)
20 19.2026-05-02 17:04:00.293.21.860186,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
21 20.2026-05-02 17:04:00.293.21.860178,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
22 21.2026-05-02 17:04:00.956.24.532266,eth0,MNF,00:00:aa:00:00:00:00:00:00:00:00:00:0.10.0.0.0.0.1.32...,42:MNF Request: HMC 00:00:aa:00:00:02 (Origin) pergunta pelo dono do IP 10.0.0.1
23 22.2026-05-02 17:04:00.957.24.532882,eth0,MNF,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.10.0.0.1.32...,42:MNF Request: HMC 00:00:aa:00:00:02 (Origin) pergunta pelo dono do IP 10.0.0.1
24 23.2026-05-02 17:04:00.958.24.531677,eth0,MNF,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.10.0.0.1.32...,42:MNF Reply: HMC 00:00:aa:00:00:02 (Origin) responde ao HMC 00:00:aa:00:00:02 (Origin)
25 24.2026-05-02 17:04:00.959.24.531634,eth0,MNF,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.10.0.0.1.32...,42:MNF Reply: HMC 00:00:aa:00:00:02 (Origin) responde ao HMC 00:00:aa:00:00:02 (Origin)
26 25.2026-05-02 17:04:00.293.25.860677,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
27 26.2026-05-02 17:04:05.890.27.41649,eth0,Ethernet,00:00:aa:00:02:33:30:00:00:05...,90:Ethernet 00:00:aa:00:02:33:30:00:05
28 27.2026-05-02 17:04:06.894.27.861086,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
29 28.2026-05-02 17:04:06.895.27.861086,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
30 29.2026-05-02 17:04:09.511.06.861274,eth0,IPV4,00:00:aa:00:02:01:00:5e:00:05:10:0:0.1.224.0.0.5...,78:IPV4 10.0.0.1 + 224.0.0.5 proto=S9
31 30.2026-05-02 17:04:11.033.32.600669,eth0,TCF,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.1.21.10.0.20.57488,00:74.TCF 10.0.1.21.57488 + 10.0.20:08 [3] + SYN (início de ligação)
32 31.2026-05-02 17:04:11.034.32.861354,eth0,TCF,00:00:aa:00:02:00:00:00:00:00:00:00:00:0.1.21.10.0.20.57488,54.TCF 10.0.20:08 + 10.0.1.21.57488 (fin.) + HST (reset)
33 32.2026-05-02 17:04:11.035.32.861354,eth0,TC
```

Ficheiro estatisticas_test.csv

O ficheiro CSV resultante foi analisado e comprovou-se que a gravação ocorre em simultâneo com a execução na consola, sem perdas de desempenho. Os campos como *timestamp*, IPs, MACs e portos ficaram corretamente tabelados.

7.6 Teste 6: Captura em Interface Física/Real

Resolução de domínio na máquina (fora do emulador) utilizando o comando `nslookup` `www.google.com`.

Comando no Host: `sudo python3 sniffer.py -i wlp0s20f3 --port 53`

```
tomas@tomas-Aspire-A315-58: ~/Desktop/3ºano 1º semestre/CC Trabalho/...
RC/Mreshark-Caseta-ro-main$ sudo python3 sniffer.py -i wlp0s20f3 --port 53
#
Bytes  Tempo(s)  Proto  Origem  Destino

=====
RC Packet Sniffer | interface: wlp0s20f3
Ctrl+C para parar

1  5.180      UDP      192.168.1.90:47291      192.168.1.254:53
74  UDP      192.168.1.90:47291 → 192.168.1.254:53      len=40
2  5.184      UDP      192.168.1.254:53       192.168.1.90:47291
138 UDP      192.168.1.254:53 → 192.168.1.90:47291      len=104
3  5.186      UDP      192.168.1.90:52895     192.168.1.254:53
74  UDP      192.168.1.90:52895 → 192.168.1.254:53      len=40
4  5.198      UDP      192.168.1.254:53       192.168.1.90:52895
298 UDP      192.168.1.254:53 → 192.168.1.90:52895      len=264
5  12.580     UDP      192.168.1.90:49669     192.168.1.254:53
75  UDP      192.168.1.90:49669 → 192.168.1.254:53      len=41
6  12.594     UDP      192.168.1.254:53       192.168.1.90:49669

(tbase) tomas@tomas-Aspire-A315-58:~$ nslookup www.google.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   www.google.com
Address: 142.251.156.119
Name:   www.google.com
Address: 142.251.150.119
Name:   www.google.com
Address: 142.251.155.119
Name:   www.google.com
Address: 142.251.151.119
Name:   www.google.com
Address: 2001:4860:482a:7700::
Name:   www.google.com
Address: 2001:4860:4828:7700::
Name:   www.google.com
Address: 2001:4860:482b:7700::
Name:   www.google.com
Address: 2001:4860:482d:7700::
Name:   www.google.com
```

Teste 6

A aplicação foi executada na placa de rede sem fios real do sistema (`wlp0s20f3`), exigindo privilégios de administrador para acesso ao tráfego. O filtro `--port 53` isolou com sucesso as interações. O *sniffer* capturou os datagramas UDP trocados entre o IP do utilizador (`192.168.1.90`) e o *gateway* local (`192.168.1.254`), reportando corretamente os comprimentos dos pacotes referentes às *queries* e às respostas.

8 Análise na Interface Real e Obstáculos Encontrados

A transposição da aplicação para uma rede física e de uso quotidiano (Fase B) foi testada com sucesso através do Teste 6, demonstrando que o *sniffer* opera com a mesma eficácia na placa de rede Wi-Fi (wlp0s20f3) como no ambiente emulado. Esta transição, embora bem-sucedida, apresentou alguns desafios técnicos que exigiram ajustes no decorrer do desenvolvimento:

- **Acesso Restrito às Interfaces:** Para que o *sniffer* conseguisse ler todo o tráfego que cruza a placa de rede real, foi necessário garantir a sua execução com privilégios de administrador (`sudo`). A tentativa de uso com um utilizador comum resultava em erros de permissão de acesso aos *sockets* do sistema operativo.
- **Elevado Ruído e Volume de Tráfego:** Numa rede doméstica, existe um fluxo constante de tráfego não intencional por parte do utilizador. Sem o uso de filtros na linha de comandos (como `--port` ou `--bpf`), a consola rapidamente ficava repleta de pacotes. Esta limitação prática justificou o desenvolvimento do modo silencioso (`--no-live`) para cenários onde se pretende apenas guardar informação num ficheiro sem "inundar" a janela do terminal.

9 Limitações da Aplicação Desenvolvida

Apesar de a ferramenta cumprir os objetivos propostos para a captura e dissecação de tráfego, o seu desenvolvimento e utilização prática revelaram algumas limitações inerentes à arquitetura adotada:

A configuração dos parâmetros de filtragem é processada exclusivamente via linha de comandos durante a inicialização do *script* principal (`sniffer.py`). Consequentemente, para alterar os critérios de filtragem, o utilizador é obrigado a interromper a captura em curso e reiniciar a aplicação com novos argumentos, não sendo possível aplicar ou remover filtros de forma dinâmica com o programa em execução.

Ao nível da Camada 7, a identificação do tráfego HTTP é efetuada de forma rudimentar e estática. O analisador baseia-se apenas na verificação das portas padrão (80/8080) e numa correspondência direta de texto (*hardcoded*) no início do *payload* bruto à procura de termos conhecidos (como `GET` ou `POST`).

A gestão de estado implementada no módulo `capture.py`, vital para relacionar pedidos e respostas ICMP e agrupar fragmentos IPv4, assenta no armazenamento de identificadores em dicionários de memória (`icmp_requests` e `ipv4_fragments`). Uma vez que não foi desenvolvido um mecanismo de limpeza periódica (*garbage collection* ou *timeouts*) para pacotes que nunca chegam a obter resposta, uma monitorização prolongada numa rede com elevado volume de tráfego resultará num aumento contínuo do consumo de RAM.

10 Conclusão

O desenvolvimento deste Trabalho Prático permitiu consolidar os conceitos teóricos lecionados. A conceção de um *packet sniffer* provou ser um desafio exigente, mas resultou numa ferramenta funcional e robusta.

A aplicação não só é capaz de capturar e dissecar tráfego em tempo real, abrangendo protocolos desde a camada de ligação de dados à camada de aplicação, como também implementa funcionalidades avançadas, indo além das funcionalidades disponibilizadas pelo *Scappy*. A introdução de gestão de estado, permitindo o agrupamento de fragmentos IPv4, a correlação de pedidos/respostas ICMP e a identificação do *handshake* TCP, junta com um motor de filtragem, eleva consideravelmente a qualidade da análise efetuada.

O emulador CORE forneceu um ambiente controlado para isolar, estimular e verificar o comportamento exato dos protocolos. Por sua vez, a transição para a interface física da máquina real testou a aplicação à prova perante volumes de tráfego intensos.

Em suma, a realização deste projeto foi muito além da simples programação, pois exigiu perceber detalhadamente como os pacotes de rede estão estruturados e como comunicam entre si, ajudando também a perceber melhor o funcionamento de uma ferramenta tão fundamental como o Wireshark.